



# HCI – intelligent *multimodal* interfaces

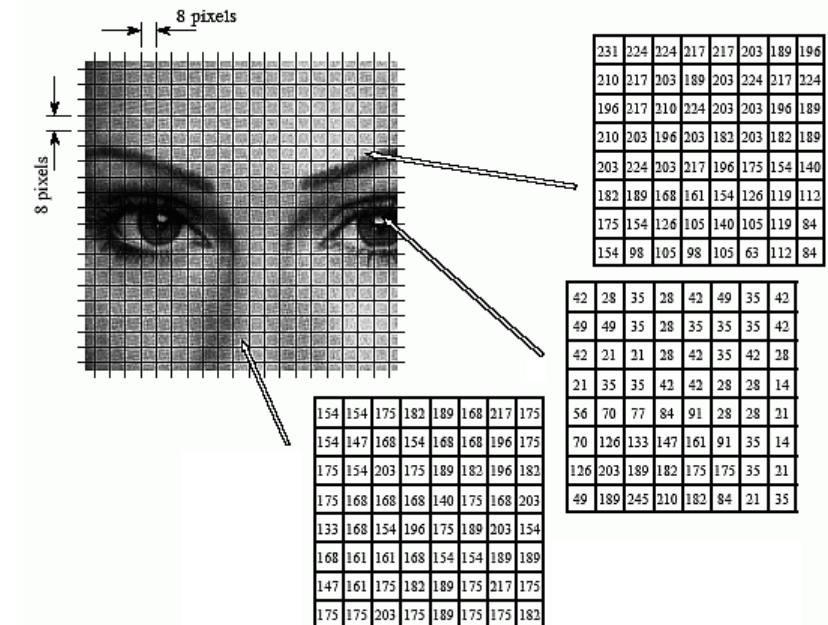
## **Computer Vision**

**Master program in Artificial Intelligence**

# Digital images

**Intensity images:** the familiar, photographlike images encoding light intensities, acquired by an optical camera

- Intensity images measure the amount of light impinging on a photosensitive device,
- A digital image is a 2D-array (matrix) of numbers



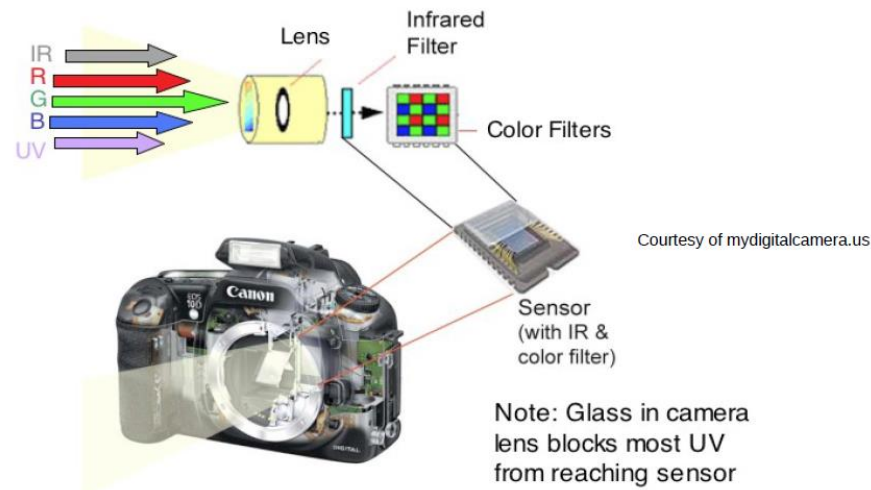
**Image Credits:** <https://www.dspguide.com/ch27/6.htm>

# Digital images

- The exact relationship of a digital image to the **physical world** is determined by the acquisition process, which depends on the **sensor used**,
- Any **information** contained in the image must be **computed** from 2-D numerical arrays, in which it is encoded.

# Optical process

- The **light rays** enter the camera through an **angular aperture** (or pupil), and hit a screen or **image plane**,
- On this plane there is a **photosensitive device** which registers light intensity,
- The most of these rays are the result of the **reflections** of the rays emitted by the light sources and hitting object surfaces



# Image focusing

- Any single point of a scene reflects light coming from possibly many directions, so that many rays reflected by the same point may enter the camera,
- All rays coming from a single scene point, **P**, must converge onto a single point **p** on the image plane (**p** is the *image* of **P**),



If this happens we say that the point **P** is *in focus*

# Optical process

To obtain an in focus image:

- **Reduce the camera's aperture** to a point, called *pinhole*. This means that only one ray from any given point can enter the camera, and create a one-to-one correspondence between visible points, rays, and image points,
- **Introduce an optical system** composed by *lenses*, *apertures*, and other elements explicitly designed to make all rays coming from the same 3-D point converge onto a single image point.



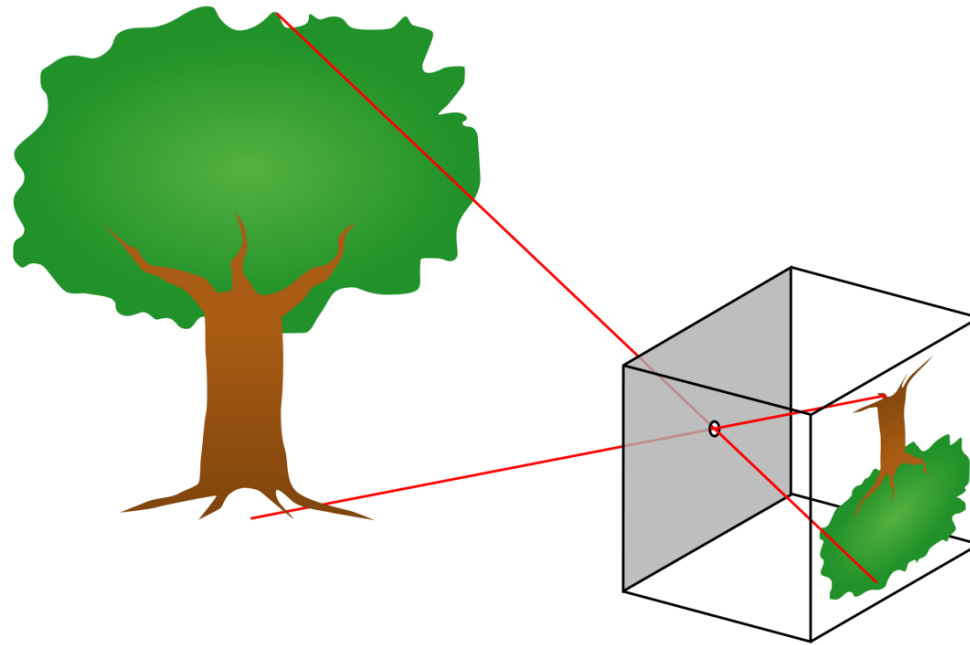
**Exposure time:** how long the image plane is allowed to receive light

# Optical process



# Image geometry

- The simplest geometric model for image acquisition is the **pinhole camera**



**Image Credits:** Wikipedia [http://en.wikipedia.org/wiki/Pinhole\\_camera](http://en.wikipedia.org/wiki/Pinhole_camera)



# Image geometry

It comes from the principle of *camera obscura* from renaissance

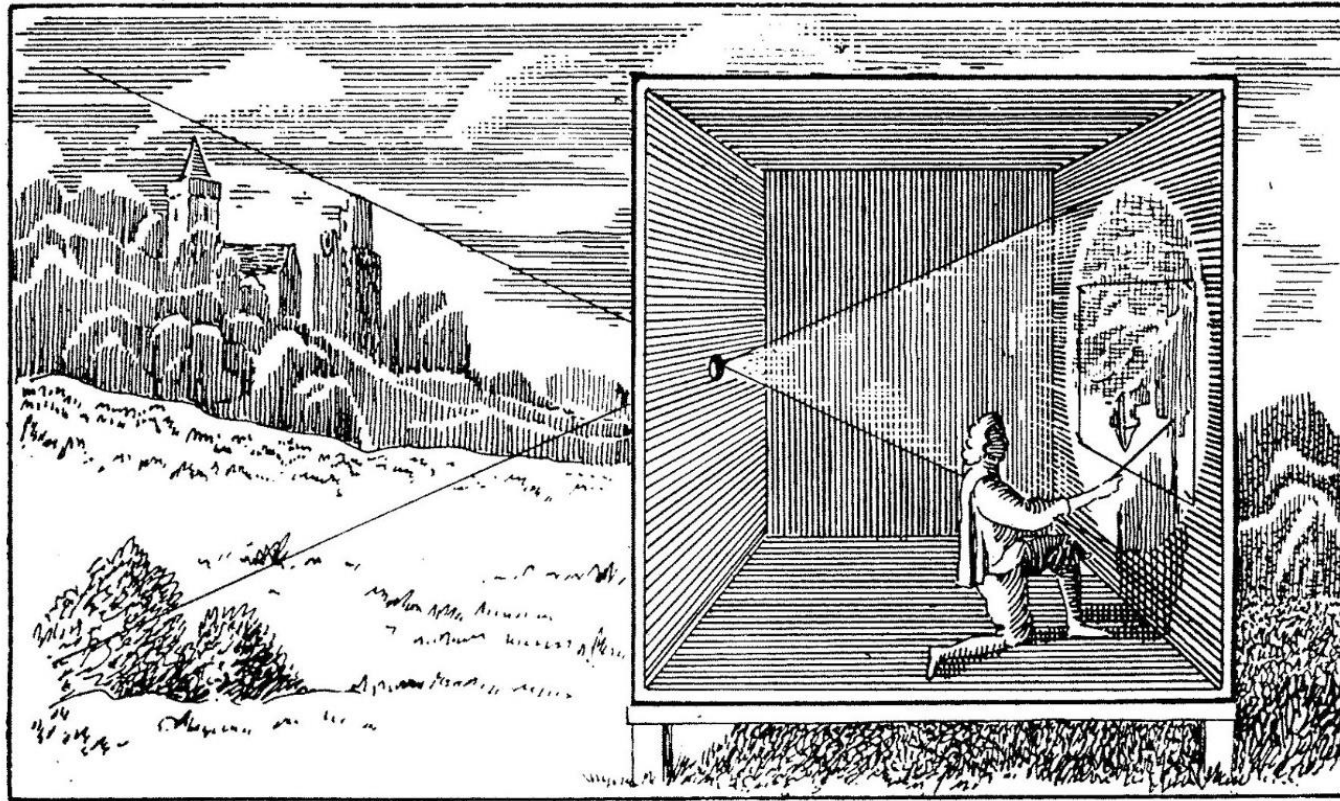
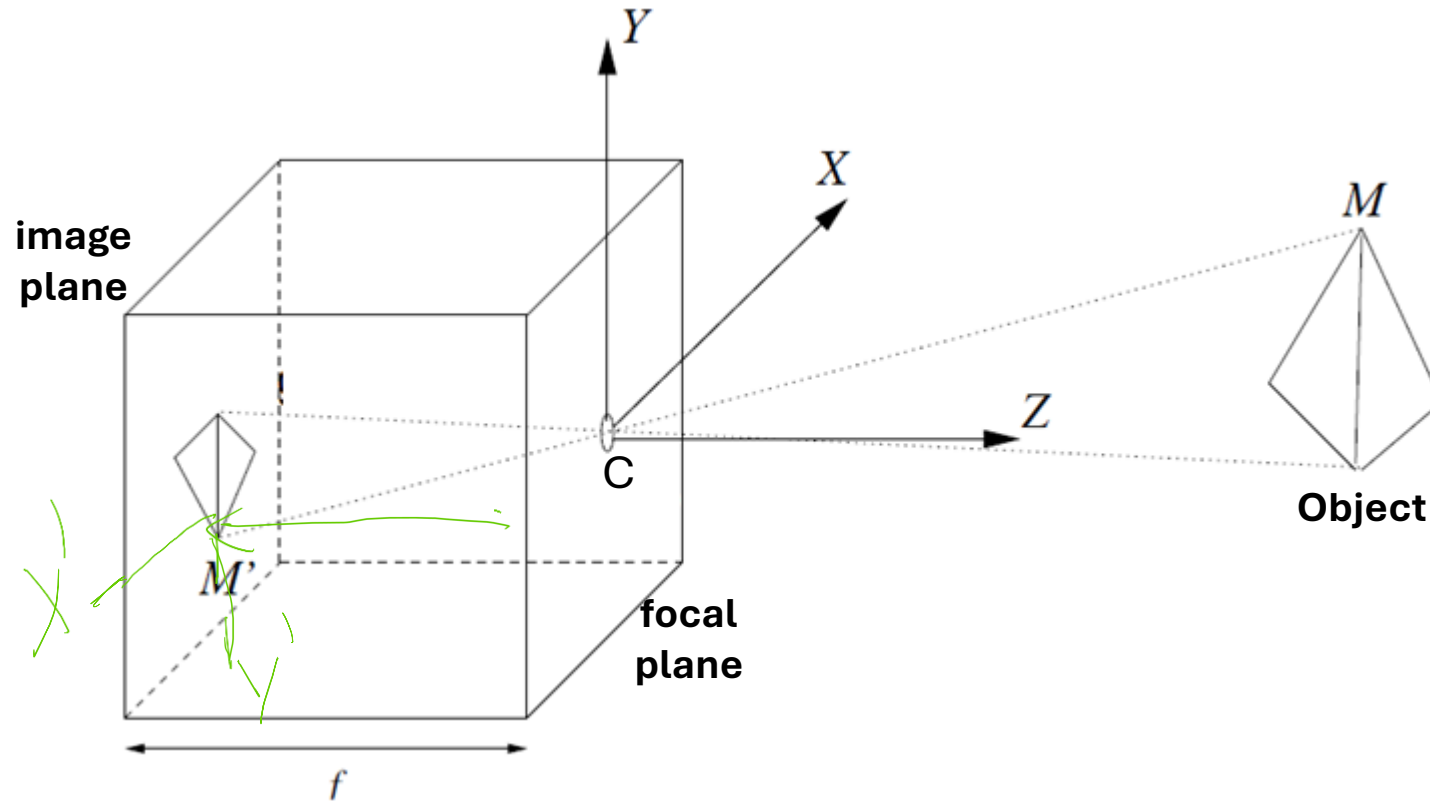


Image credits: <http://www.arabeschi.it/>

# Image geometry



# Image geometry

- Ingredients:
  - The image is created on the **image plane** (or **retina plane**),
  - The hole C where the light enters the camera is the **projection center** or **point of view** or **focus of projection**,
  - $f$  is the **focal length**, i.e., the distance between the image plane and C
  - The plane where C is lying that is parallel to the image plane is called ***focal plane***,

**N.B.** Here the focal length is related to the image plane and not to the lens. This focal length and the focal length of the lens are two different quantities.

# Image geometry

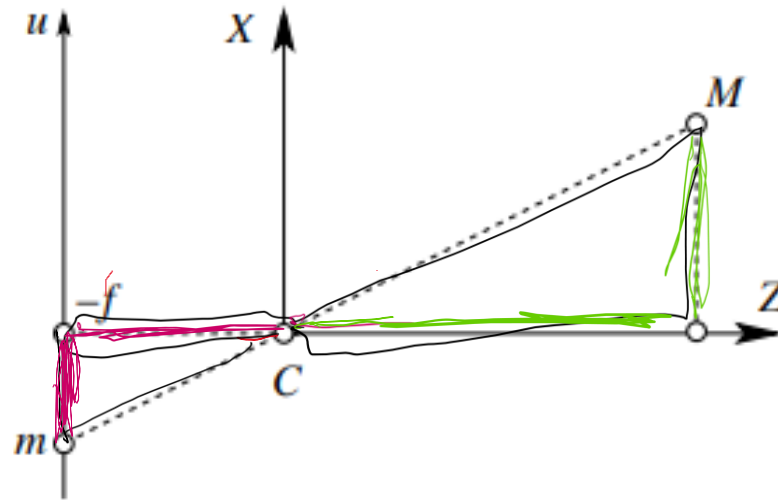


Fig. 4.2. Vista semplificata del modello della fotocamera. I due triangoli che hanno per ipotenusa  $\overline{CM}$  e  $\overline{Cm}$  sono simili tra loro.

# Image geometry

- Equations:

$$\frac{-X'}{f} = \frac{X}{Z} \text{ and } \frac{-Y'}{f} = \frac{Y}{Z}$$

$$X' = \frac{-fX}{Z} \quad Y' = \frac{-fY}{Z} \quad Z' = -f$$

These equations define the **projective perspective** image acquisition process.

# Image geometry

- Equations:

$$\frac{-X'}{f} = \frac{X}{Z} \quad \text{e} \quad \frac{-Y'}{f} = \frac{Y}{Z}$$

$$X' = -f \frac{X}{Z} \quad Y' = -f \frac{Y}{Z} \quad Z' = -f$$

The denominator term  $Z$  is the cause of the **perspective effect**: the dimension of the projection of an object to the image depends by its distance to the observer.

# Image geometry

- When the observed object is very thin, with respect to its distance from the camera, we can approximate the projective perspective with orthographic projection (scaled) or ***weack perspective***.
  - When the depth  $Z$  is given by  $Z_0 \pm \Delta Z$  con  $(\Delta Z/Z_0) \ll 1$  the perspective factor  $f/Z$  can be approximated by the constant  $f/Z_0$

$$X' = \frac{-f}{Z_0} X \quad Y' = \frac{-f}{Z_0} Y$$

This is an orthographic projection compoled by a scale of a scale factor  $f/Z_0$

# Image geometry



**Perspective image:** lines are converging.



**Orthographic image:** the distance to the observer is much larger than the size of the objects in the scene.



# Digital images

- In a digital camera the image plane is composed by a CCD (Charge-Coupled Device) or a CMOS (Complementary Metal-Oxide Semiconductor) matrix,
  - This matrix contains  $n \times m$  elements of rectangular photosensor cells. Each photosensor cell converts light energy into voltage.
  - The video signal is digitized into a  $N \times M$  matrix of integer values (eg., in  $[0, 255]$ ), i.e., the digital image.
  - The element of this matrix are called *pixel* (picture element) and  $N \times M$  is the image size.

# Digital images

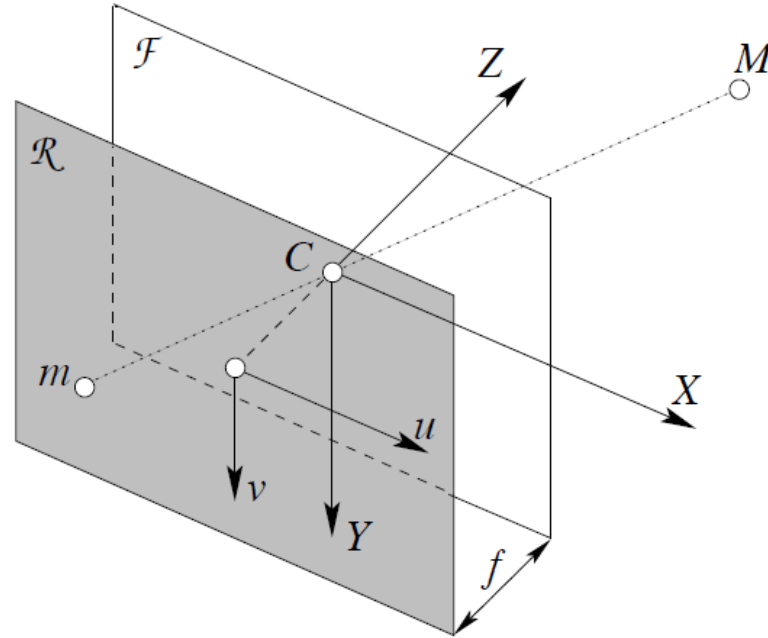
- In general, the intensity value of the image pixel is given by  $I(u,v)$
- The coordinate system  $u,v$  has the origin on the top-left of the image,
- The CCD dimension  $n \times m$  is not necessary the same as the image size  $N \times M$ :

$$u_{\text{pix}} = \frac{N}{n} u_{\text{CCD}} \qquad v_{\text{pix}} = \frac{M}{m} v_{\text{CCD}}$$

Since the CCD elements have a physical dimension (i.e., micron), it is possible to measure the so called *effective horizontal and vertical sizes* of the pixel (in meter).

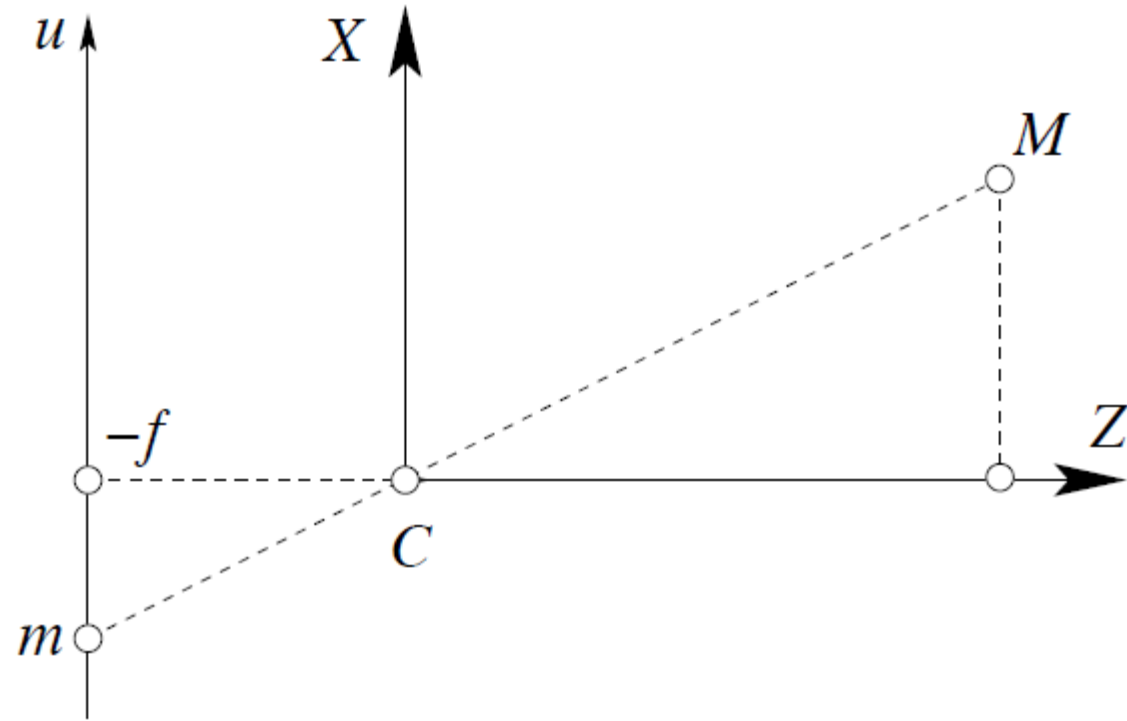
# Camera calibration

# Pinhole camera



**Image Credits:** Andrea Fusiello

# Pinhole camera



**Image Credits:** Andrea Fusiello

Now consider a point  $M$  of coordinates  $\tilde{\mathbf{M}} = [X, Y, Z]^T$  in 3D space and let  $m$  of coordinates  $\tilde{\mathbf{m}} = [u, v]^T$  be its projection onto  $\mathcal{Q}$  through  $C$ . In the following, unless otherwise noted, we will always use  $\sim$  to denote Cartesian coordinates and distinguish them from homogeneous coordinates.

By simple considerations on the similitude of triangles (Fig. 3.3), we arrive at the following relation:

$$\frac{f}{Z} = \frac{-u}{X} = \frac{-v}{Y} \quad (3.1)$$

that is

$$\begin{cases} u = \frac{-f}{Z} X \\ v = \frac{-f}{Z} Y. \end{cases} \quad (3.2)$$

This is the *perspective projection*. The transformation in Cartesian coordinates is clearly non-linear because of the division by  $Z$ . Using homogeneous coordinates<sup>1</sup> instead it becomes *linear*.

Let therefore be

$$\mathbf{m} = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \quad \text{and} \quad \mathbf{M} = \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (3.3)$$

the homogeneous coordinates of  $m$  and  $M$ , respectively. Note that by setting the last coordinate to 1, we have excluded the points at infinity (to include them we would

---

<sup>1</sup> The homogeneous coordinates of a 2D point in the image are the  $(x_1, x_2, x_3)$ , where  $(x_1/x_3, x_2/x_3)$  are the corresponding Cartesian coordinates. Thus, there is a one-to-many correspondence between Cartesian and homogeneous coordinates. Homogeneous coordinates can represent any point in the Euclidean plane and also points at infinity, which have the third component equal to zero and therefore do not have a corresponding Cartesian representation. See [Appendix D](#).

have used a generic last component). So the equation for perspective projection, in this simplified case, rewrites:

$$Z \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = Z \begin{bmatrix} -fX/Z \\ -fY/Z \\ 1 \end{bmatrix} = \begin{bmatrix} -fX \\ -fY \\ Z \end{bmatrix} = \underbrace{\begin{bmatrix} -f & 0 & 0 & 0 \\ 0 & -f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}}_P \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}. \quad (3.4)$$

Switching to matrix notation:

$$\mathbf{m} = \frac{1}{Z} P \mathbf{M} \quad (3.5)$$

or, also:

$$\mathbf{m} \simeq P \mathbf{M} \quad (3.6)$$

where  $\simeq$  means “equal up to a scaling factor”.

The matrix  $P$  represents the geometric model of the camera and is called Perspective Projection Matrix (**PPM**) or *camera matrix*.

Listing 3.1 reports the implementation of a generic projective transformation, which includes perspective projection as a special case.



# Pinhole camera

More generic model: what happen in the camera?

**Observation 1:** in standard image the image coordinates are centered on the upper-left corner

$$\begin{cases} u = \frac{-f}{z}x + u_0 \\ v = \frac{-f}{z}y + v_0 \end{cases} ,$$

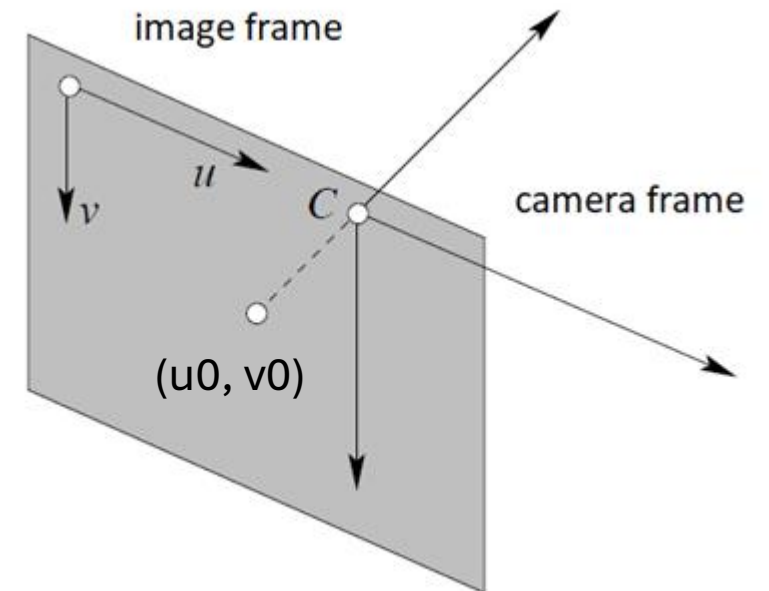


Image Credits: Andrea Fusiello

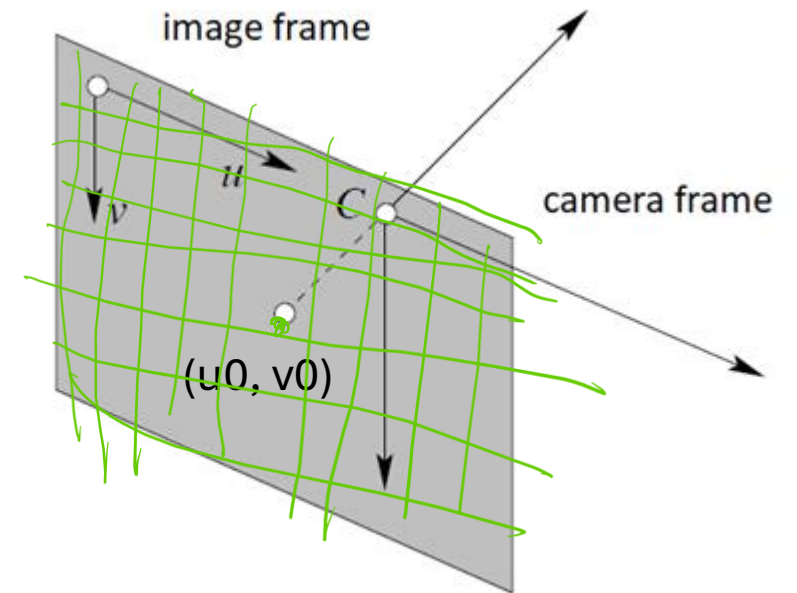
# Pinhole camera

**More generic model: what happen in the camera?**

**Observation 2:** when point is projected to the image plane it generates a pixel on the digital image, according to the image digitization process.

- We need to change the measurement system, i.e., from point(meter) to pixel in the pixel space,
- We need to involve the inverse of the effective pixel dimension  $k_u$  and  $k_v$  along x and y axis directions (pixel  $\times m^{-1}$ )

$$\begin{cases} u = k_u \frac{-f}{z} x + u_0 \\ v = k_v \frac{-f}{z} y + v_0 \end{cases} ,$$



**Image Credits:** Andrea Fusiello

### 3.4.1 Intrinsic Parameters

The transformation from the sensor coordinates to pixels is modelled by an affine transformation  $A$  that takes into account the translation of the origin at the upper left corner, the independent rescaling of the  $u$  and  $v$  axes for changing the unit of measurement from metres to pixels and the reversal of the direction of the axes:

$$A = \begin{bmatrix} -k_u & 0 & u_0 \\ 0 & -k_v & v_0 \\ 0 & 0 & 1 \end{bmatrix} \quad (3.7)$$

where  $(u_0, v_0)$  are the coordinates in pixels of the principal point and  $k_u$  ( $k_v$ ) is the inverse of the *effective pixel size* (see Sect. 2.3) along the direction  $u$  ( $v$ ), measured in  $\text{m}^{-1}$ . The matrix  $A$  transforms the coordinates from metres to pixels; hence, it pre-multiplies the [PPM](#), which becomes:

$$P = \begin{bmatrix} -k_u & 0 & u_0 \\ 0 & -k_v & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -f & 0 & 0 & 0 \\ 0 & -f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} fk_u & 0 & u_0 & 0 \\ 0 & fk_v & v_0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} = K[I_3, \mathbf{0}] \quad (3.8)$$

where

$$K = \begin{bmatrix} f k_u & 0 & u_0 \\ 0 & f k_v & v_0 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{and} \quad [I_3, \mathbf{0}] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}. \quad (3.9)$$

$K$  is the *matrix of intrinsic parameters*, while  $[I_3, \mathbf{0}]$  encodes the essence of the perspective transformation, without any parameters. It is obtained when the world reference frame coincides with the camera reference frame and the image points are expressed in *normalised image coordinates*, that is,

$$\mathbf{q} = K^{-1} \mathbf{m}. \quad (3.10)$$

Note that whereas pixel coordinates are always measurable in the image, to step to normalised image coordinates the matrix of intrinsic parameters is needed.

Some authors parameterise  $K$  with its entries by introducing the focal length expressed in vertical pixels  $\alpha_v$  and the skew parameter  $s$ :

$$K = \begin{bmatrix} \alpha_u & s & u_0 \\ 0 & \alpha_v & v_0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (3.13)$$

# Intrinsic parameters

- Intrinsic parameters are: principal point, focal length in pixel along  $u$  and along  $v$  (there is further parameter  $s$  but not relevant here)...

# Pinhole camera

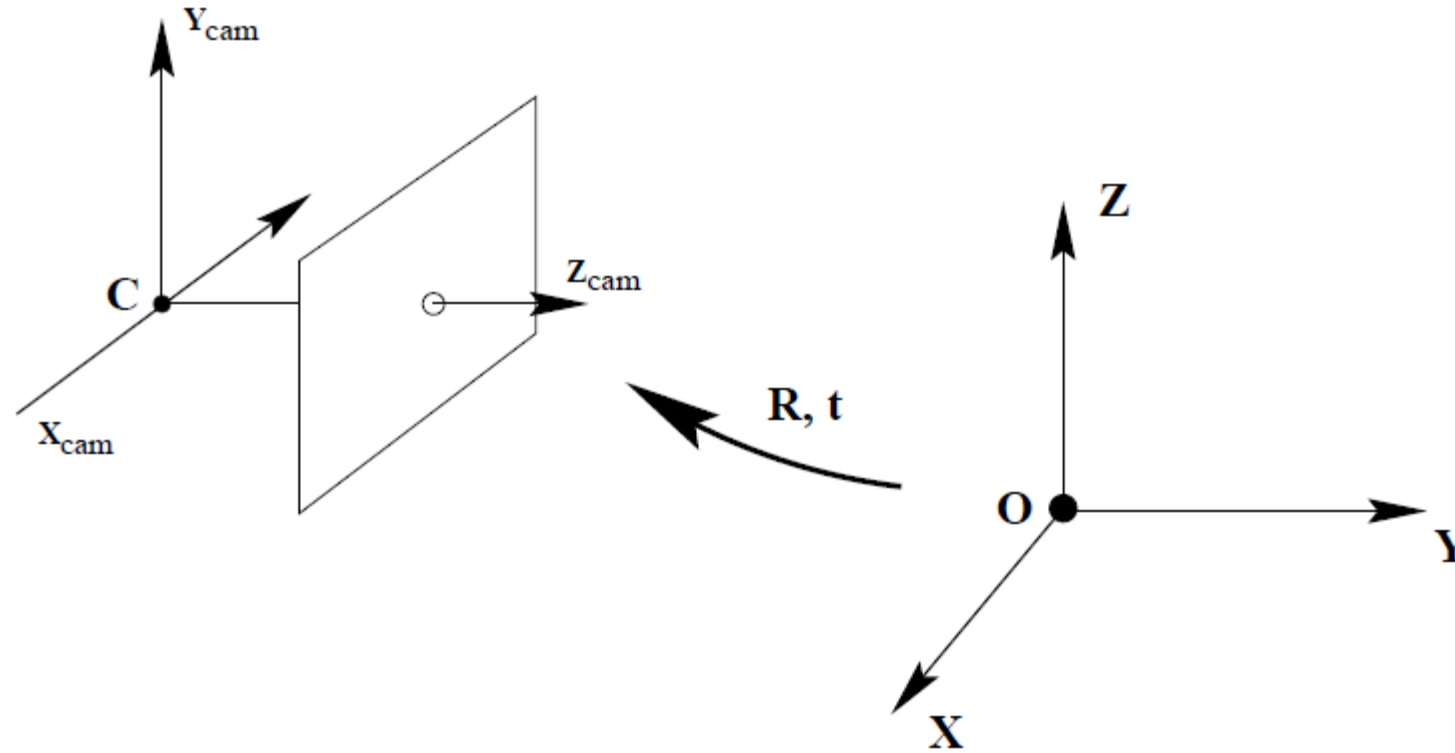


Fig. 6.3. *The Euclidean transformation between the world and camera coordinate frames.*

# Extrinsic parameters

They involve the rotation (encoded by 3 parameters) and translation (3 parameters) parameters to bring points from world space to camera space

# Full perspective matrix

Depending on convenience, we may also consider the following expression for the PPM:

$$P = K[R, \mathbf{t}] \quad (3.21)$$

which is obtained by substituting into (3.20) the block form of  $G$  and developing the product with the central matrix.

By letting

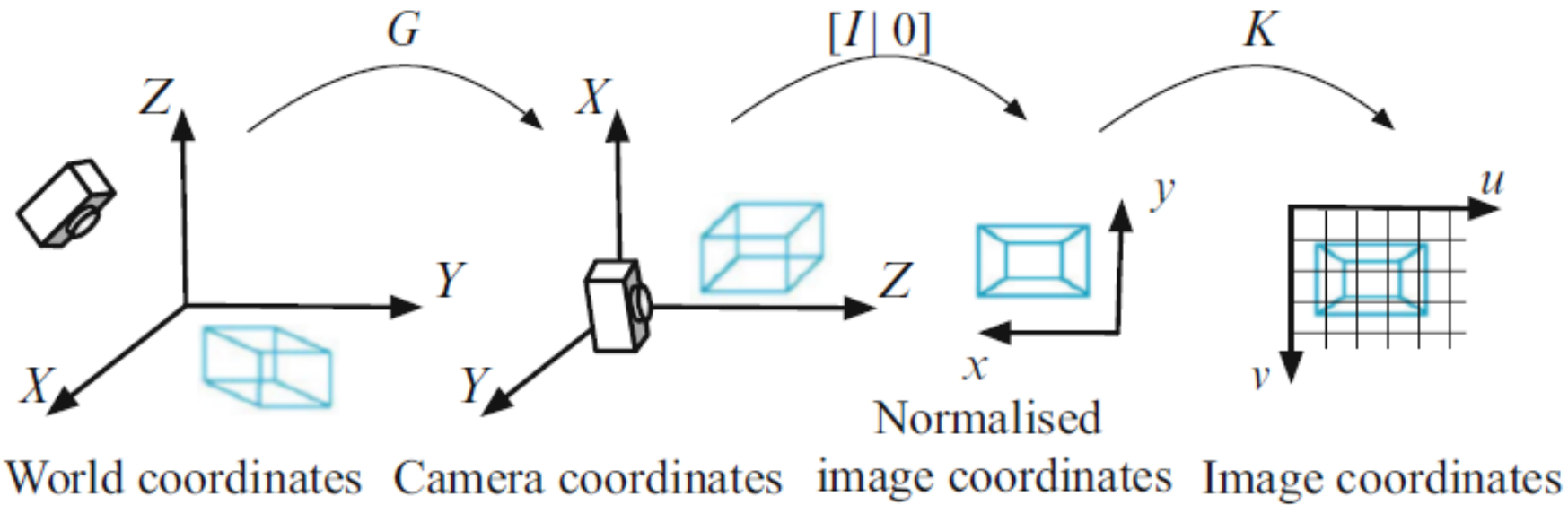
$$\mathbf{t} = \begin{bmatrix} t_1 \\ t_2 \\ t_3 \end{bmatrix} \quad \text{e} \quad R = \begin{bmatrix} \mathbf{r}_1^\top \\ \mathbf{r}_2^\top \\ \mathbf{r}_3^\top \end{bmatrix} \quad (3.22)$$

we get the following expression for  $P$  as a function of the elements of the matrices  $K$  and  $G$ :

$$P = \begin{bmatrix} \alpha_u \mathbf{r}_1^\top + u_0 \mathbf{r}_3^\top & \alpha_u t_1 + u_0 t_3 \\ \alpha_v \mathbf{r}_2^\top + v_0 \mathbf{r}_3^\top & \alpha_v t_2 + v_0 t_3 \\ \mathbf{r}_3^\top & t_3 \end{bmatrix}. \quad (3.23)$$



# Involved transformation



**Fig. 3.7** Coordinate systems involved in camera modelling

# Camera calibration method

- **Direct method:**

See notes to estimate the full Perspective Projection Camera camera:

$$\underbrace{\begin{bmatrix} \overset{1 \times 4}{M_1^T} & \overset{4}{0} & \overset{1 \times 4}{-u_1 M_1^T} \\ 0 & M_1^T & -v_1 M_1^T \\ \vdots & \vdots & \vdots \\ \overset{1 \times 4}{M_n^T} & 0 & -u_n M_n^T \\ 0 & M_n^T & -v_n M_n^T \end{bmatrix}}_A \underbrace{\begin{bmatrix} \overset{12 \times 1}{P_1} \\ P_2 \\ P_3 \end{bmatrix}}_X = \underbrace{\begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix}}_0$$

# Practical variation:

- The setting of the homogeneous linear system can be defined in practice using a combination of algebraic operators:
  - **Kronecker's product**,
  - **Cross product** and **Matrix of cross product** with given vector,
  - **Vectorization**, and **vec-trick**

# Kronecker' product

## A.14 Kronecker's Product

**Definition A.20 (Kronecker's Product)** Let  $A$  be a  $m \times n$  matrix and  $B$  a  $p \times q$  matrix. Kronecker's product<sup>3</sup> of  $A$  and  $B$  is the matrix  $mp \times nq$  defined by

$$A \otimes B = \begin{bmatrix} a_{11}B & \dots & a_{1n}B \\ \vdots & & \vdots \\ a_{m1}B & \dots & a_{mn}B \end{bmatrix}. \quad (\text{A.56})$$

Note that Kronecker's product is defined for any pair of matrices. Properties:

1.  $(A \otimes B) \otimes C = A \otimes (B \otimes C)$
2.  $(A \otimes B)(C \otimes D) = AC \otimes BD$  if the dimensions are compatible
3.  $A \otimes B \neq B \otimes A$
4.  $(A \otimes B)^{\top} = A^{\top} \otimes B^{\top}$
5.  $(A \otimes B)^{-1} = A^{-1} \otimes B^{-1}$ .

# Cross product

## A.13 Cross Product

This is a product between two vectors that returns a vector, and is defined only in  $\mathbb{R}^3$ .

**Definition A.19 (Cross Product)** The cross product of two vectors  $\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$  is defined as the vector:

$$\mathbf{a} \times \mathbf{b} = \begin{bmatrix} \det \begin{bmatrix} a_2 & a_3 \\ b_2 & b_3 \end{bmatrix} \\ -\det \begin{bmatrix} a_1 & a_3 \\ b_1 & b_3 \end{bmatrix} \\ \det \begin{bmatrix} a_1 & a_2 \\ b_1 & b_2 \end{bmatrix} \end{bmatrix}. \quad (\text{A.48})$$

The cross product is used (among other things) to check whether two vectors differ only by a multiplicative constant.

*Remark A.14* Given two vectors  $\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$ ,  $\mathbf{a}, \mathbf{b} \neq \mathbf{0}$ , we have:  $\mathbf{a} \times \mathbf{b} = \mathbf{0} \iff \mathbf{a} = \lambda \mathbf{b}, \lambda \in \mathbb{R}$ .

In fact, the hypothesis is equivalent to stating that all determinants of order two extracted from  $[\mathbf{a}, \mathbf{b}]$  are null, so it has rank one, from which the thesis follows, and vice versa.

# Matrix of a cross product

## A.13 Cross Product

The cross product is naturally associated with an antisymmetric matrix:

*Remark A.15* Given a vector  $\mathbf{a} \in \mathbb{R}^3$ , the matrix

$$[\mathbf{a}]_{\times} = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix} \quad (\text{A.49})$$

acts as the cross product for  $\mathbf{a}$ , that is,  $[\mathbf{b}]_{\times} \mathbf{b} = \mathbf{a} \times \mathbf{b}$ .

The matrix  $[\mathbf{a}]_{\times}$  is antisymmetric, and  $\ker([\mathbf{a}]_{\times}) = \mathbf{a}$ , since  $[\mathbf{a}]_{\times} \mathbf{a} = \mathbf{a} \times \mathbf{a} = \mathbf{0}$ . It is therefore singular, and in particular it has rank two, since the rank must be even (Remark [A.10](#)).

# Vectorization and vec-trick

**Definition A.21 (Vectorisation)** The vectorisation of a  $m \times n$  matrix  $A$ , denoted by  $\text{vec } A$ , is the vector  $mn \times 1$  that is obtained by stacking the columns of  $A$  one below the other.<sup>4</sup>

The connection between the Kronecker product and vectorisation is given by the following relation, also known as the *vec-trick*:

$$\text{vec}(AXB) = (B^T \otimes A) \text{vec } X \quad (\text{A.59})$$

for matrices  $A, B, X$  of compatible dimensions. This will be very useful for extracting the unknown  $X$  from a matrix equation.

# Direct method revised:

Given the correspondences between  $n$  *non-coplanar* control points  $\mathbf{M}_i$  and their projections in the image  $\mathbf{m}_i$ , it is required to compute a matrix  $P$  such that:

$$\mathbf{m}_i \simeq P\mathbf{M}_i \quad i = 1 \dots n. \quad (4.1)$$

To eliminate the scaling factor from (4.1) we exploit the cross product, rewriting the previous equation as:

$$\mathbf{m}_i \times P\mathbf{M}_i = \mathbf{0}. \quad (4.2)$$

Then, using the properties of the Kronecker product (Sect. A.14), and in particular the “vec trick” (A.59), we derive that:

$$\begin{aligned} \mathbf{m}_i \times P\mathbf{M}_i = \mathbf{0} &\iff [\mathbf{m}_i]_{\times} P\mathbf{M}_i = \mathbf{0} \iff \\ \text{vec}([\mathbf{m}_i]_{\times} P\mathbf{M}_i) = \mathbf{0} &\iff (\mathbf{M}_i^{\top} \otimes [\mathbf{m}_i]_{\times}) \text{vec } P = \mathbf{0}. \end{aligned} \quad (4.3)$$



# Direct method revised:

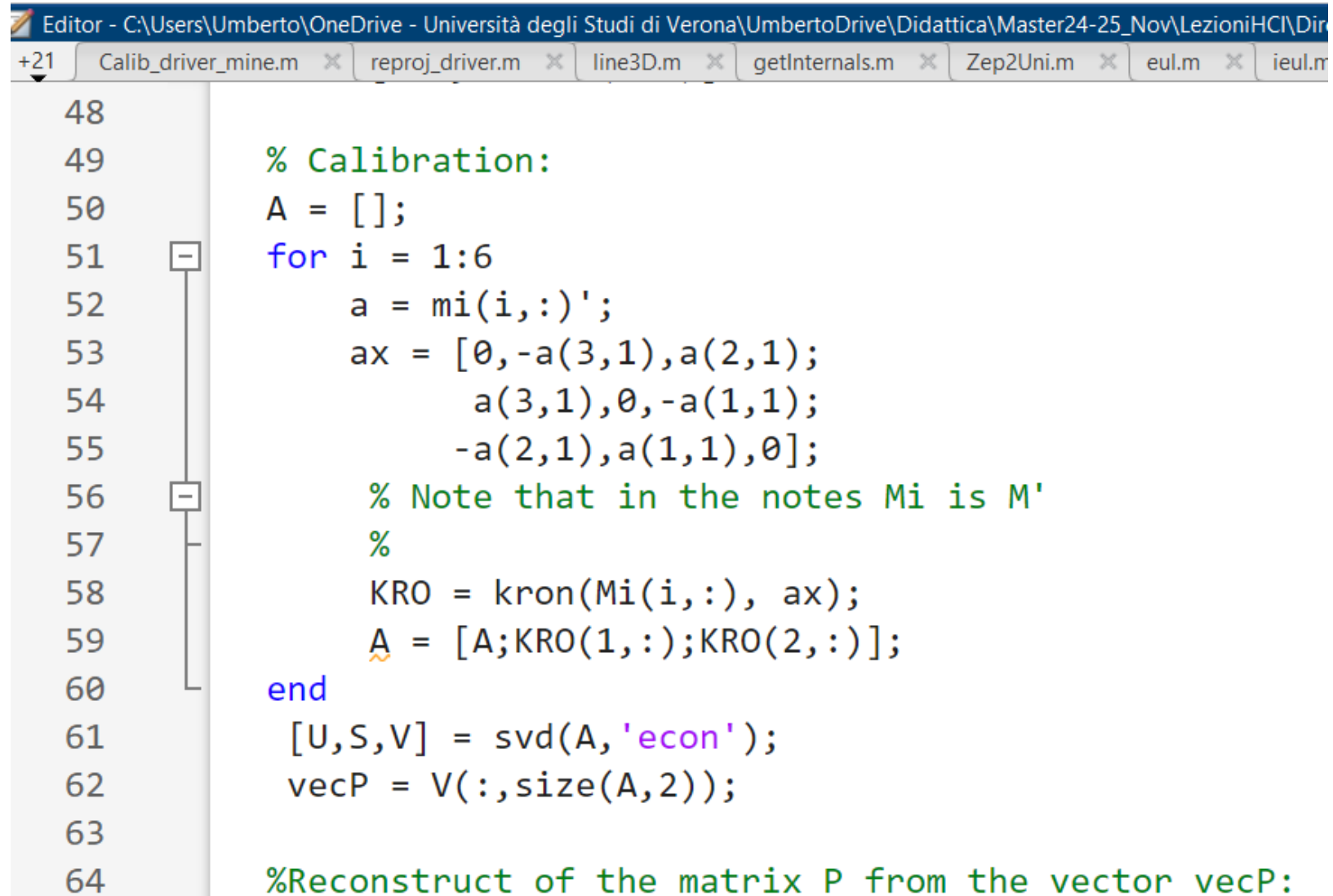
These are three equations in 12 unknowns, but only 2 of them are independent; in fact, the rank of  $(\mathbf{M}_i^T \otimes [\mathbf{m}_i]_\times)$  is two since it is the Kronecker product of a matrix of rank one with a matrix of rank two.

With  $n$  points we obtain a system of  $2n$  homogeneous linear equations, which we can write in matrix form as

$$A \operatorname{vec} P = \mathbf{0} \quad (4.4)$$

where  $A$  is the  $2n \times 12$  matrix of coefficients and depends on the coordinates of the control points, while the vector of unknowns  $\operatorname{vec} P$  contains the 12 elements of  $P$  read by columns.

# See Matlab code:

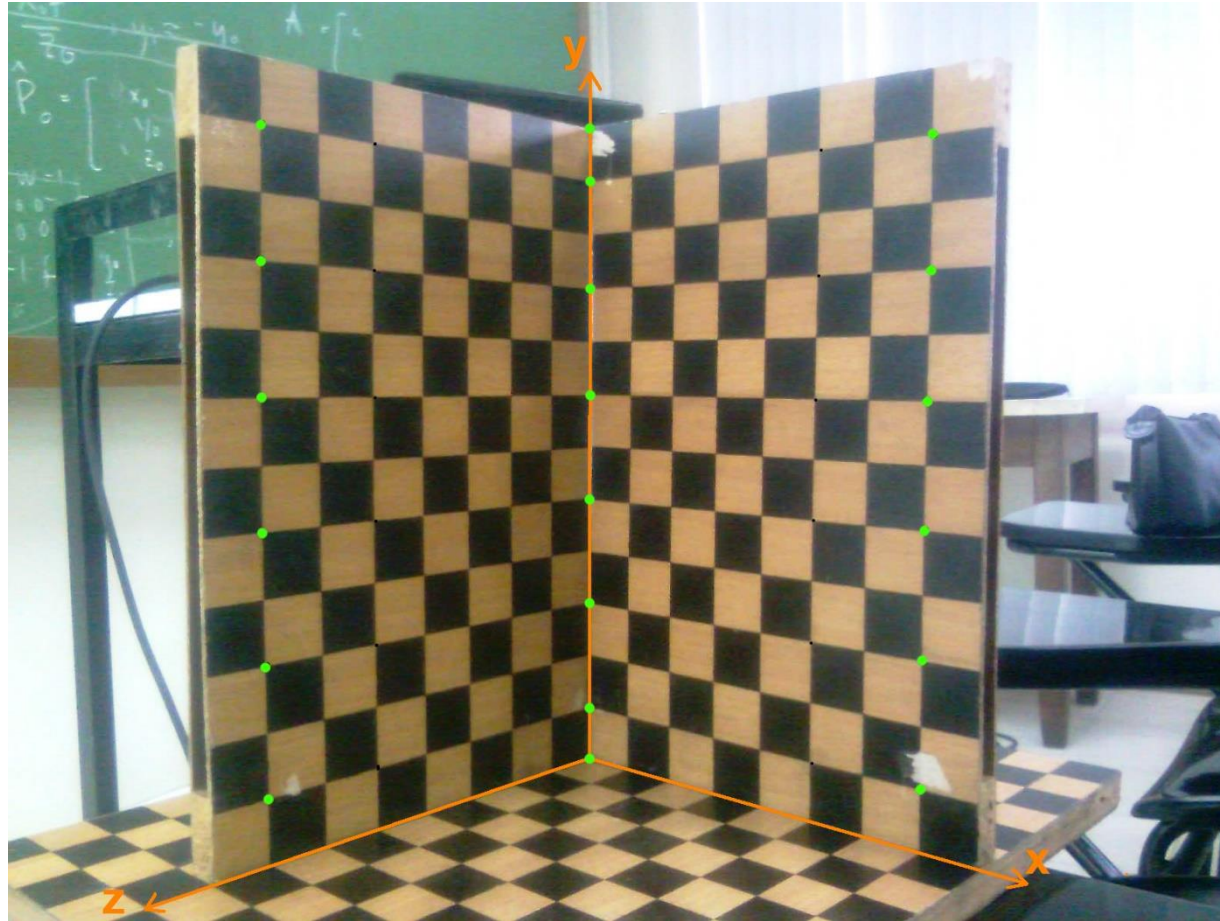


The screenshot shows a MATLAB editor window with the following code:

```
Editor - C:\Users\Umberto\OneDrive - Università degli Studi di Verona\UmbertoDrive\Didattica\Master24-25_Nov\LezioniHCI\Dir
+21 Calib_driver_mine.m x reproj_driver.m x line3D.m x getInternals.m x Zep2Uni.m x eul.m x ieul.m

48
49 % Calibration:
50 A = [];
51 for i = 1:6
52     a = mi(i,:);
53     ax = [0,-a(3,1),a(2,1);
54          a(3,1),0,-a(1,1);
55          -a(2,1),a(1,1),0];
56     % Note that in the notes Mi is M'
57     %
58     KRO = kron(Mi(i,:), ax);
59     A = [A;KRO(1,:);KRO(2,:)];
60 end
61 [U,S,V] = svd(A,'econ');
62 vecP = V(:,size(A,2));
63
64 %Reconstruct of the matrix P from the vector vecP:
```

# Calibration object example



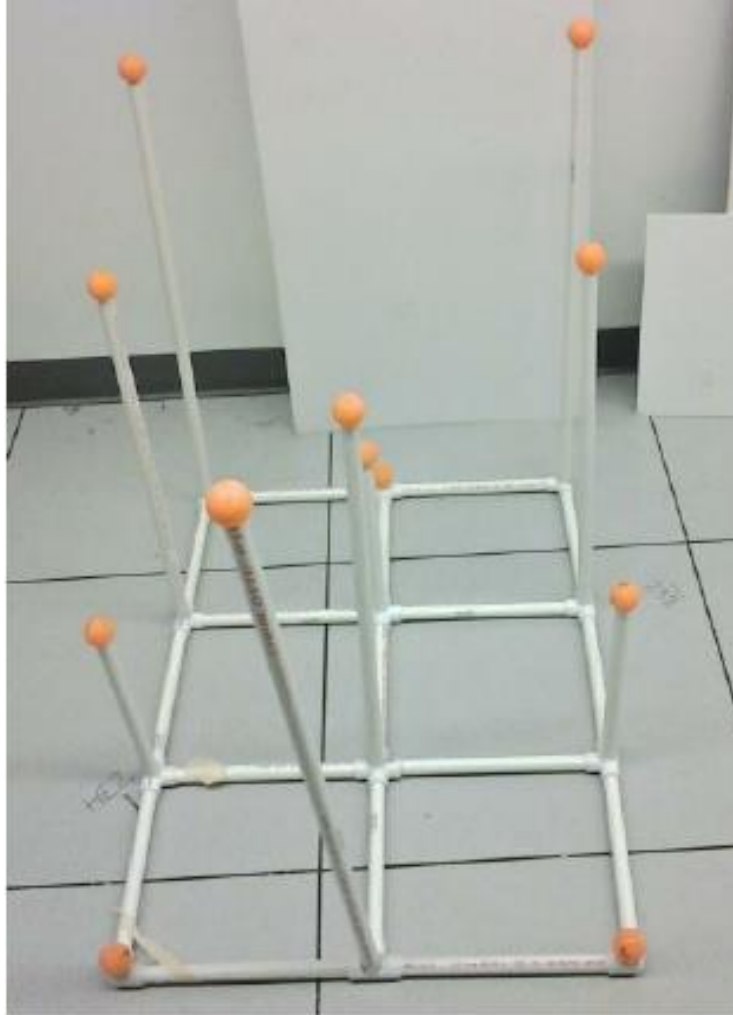
# Calibration object example



# Calibration object example



# Calibration object example

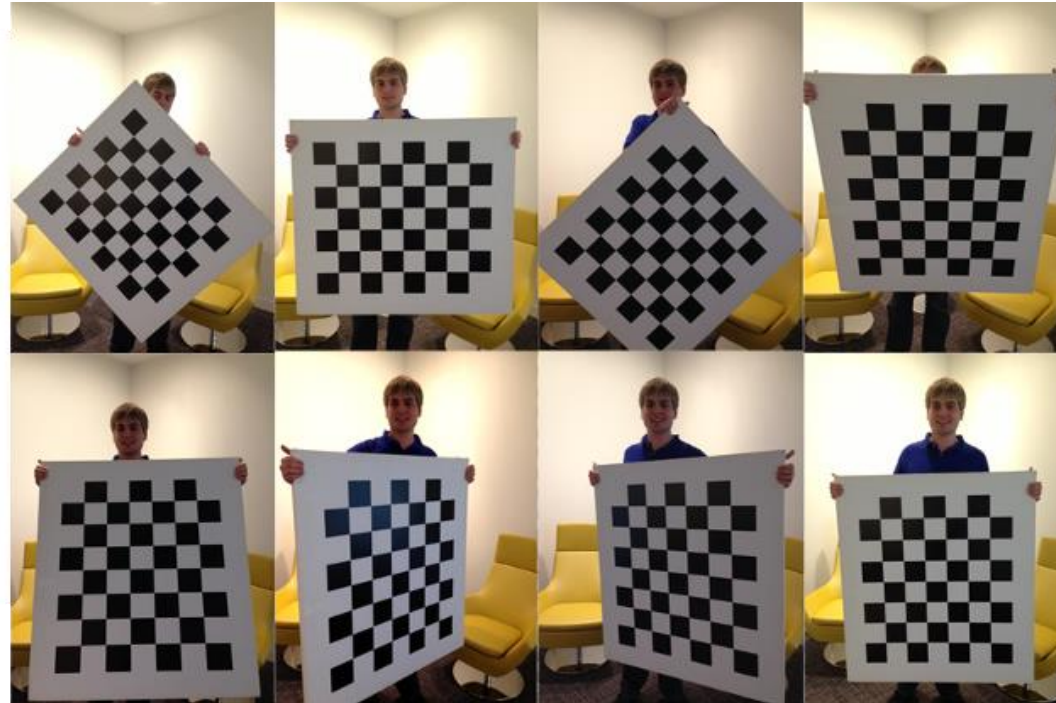
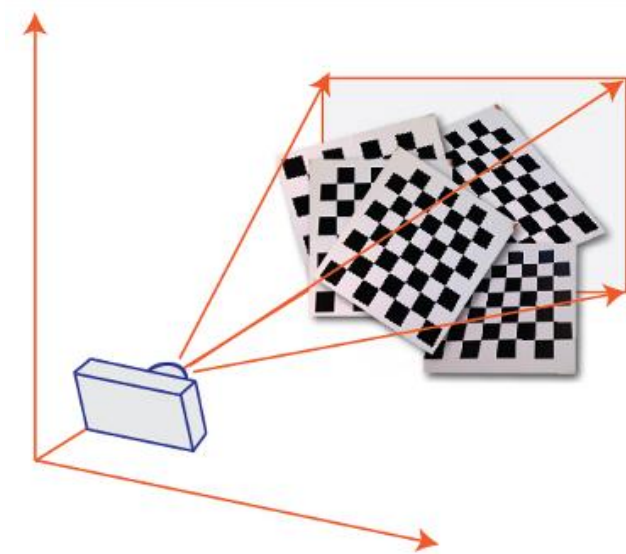




# Calibration object example

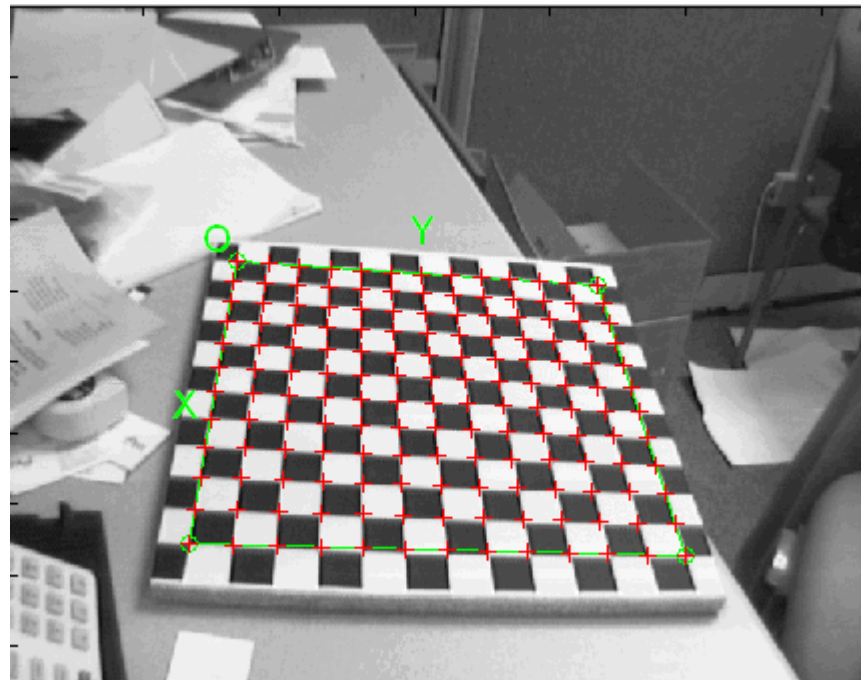


# Calibration object example





# Calibration object example



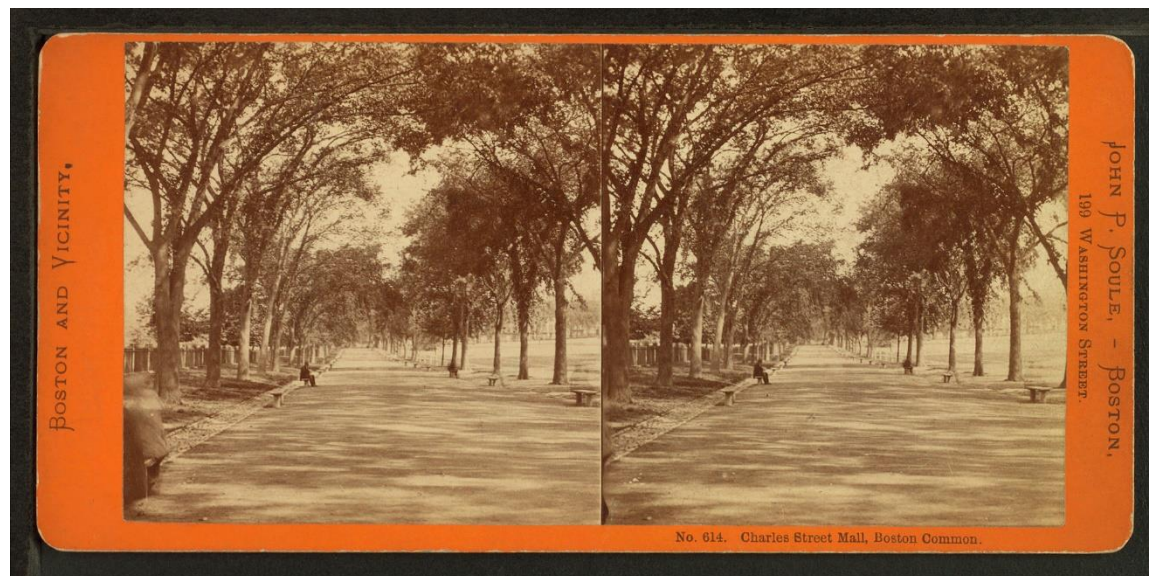
# Triangulation

# Stereopsi

What is stereopsis? Because our two eyes are located in different positions in the head, their views of a 3-D scene are slightly disparate. One can easily experience directly this binocular disparity by looking at objects not too distant and noting their different relative positions when closing each eye in turn. The disparity of each "point" depends on its distance from the fixation point of the two eyes. Our brain is capable of measuring this disparity and using it to produce the sensation of depth that is the subjective estimate of relative distance. This is stereopsis and its sole basis is the horizontal disparity between the two retinal images.

«THE ANALYSIS OF STEREOPSIS»Poggio&Poggio1984

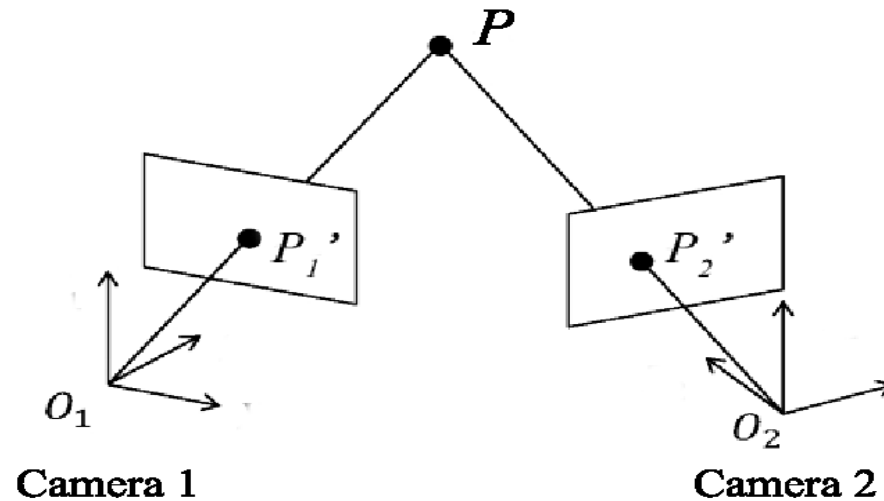
# Stereopsi



<https://en.wikipedia.org/wiki/Stereoscopy>

# Stereopsi

- In computer vision **stereopsi** is the process of the estimation of 3D structure of a scene from a **pair of images** that observe the scene from **two point of view**.

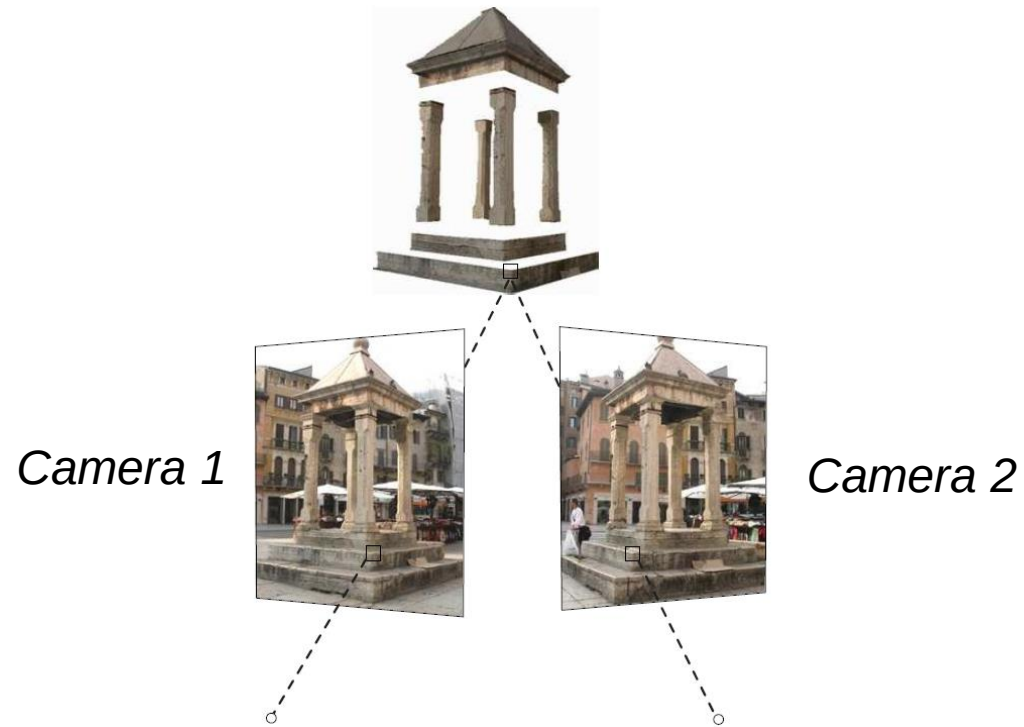


# Stereopsi

## Two main problems:

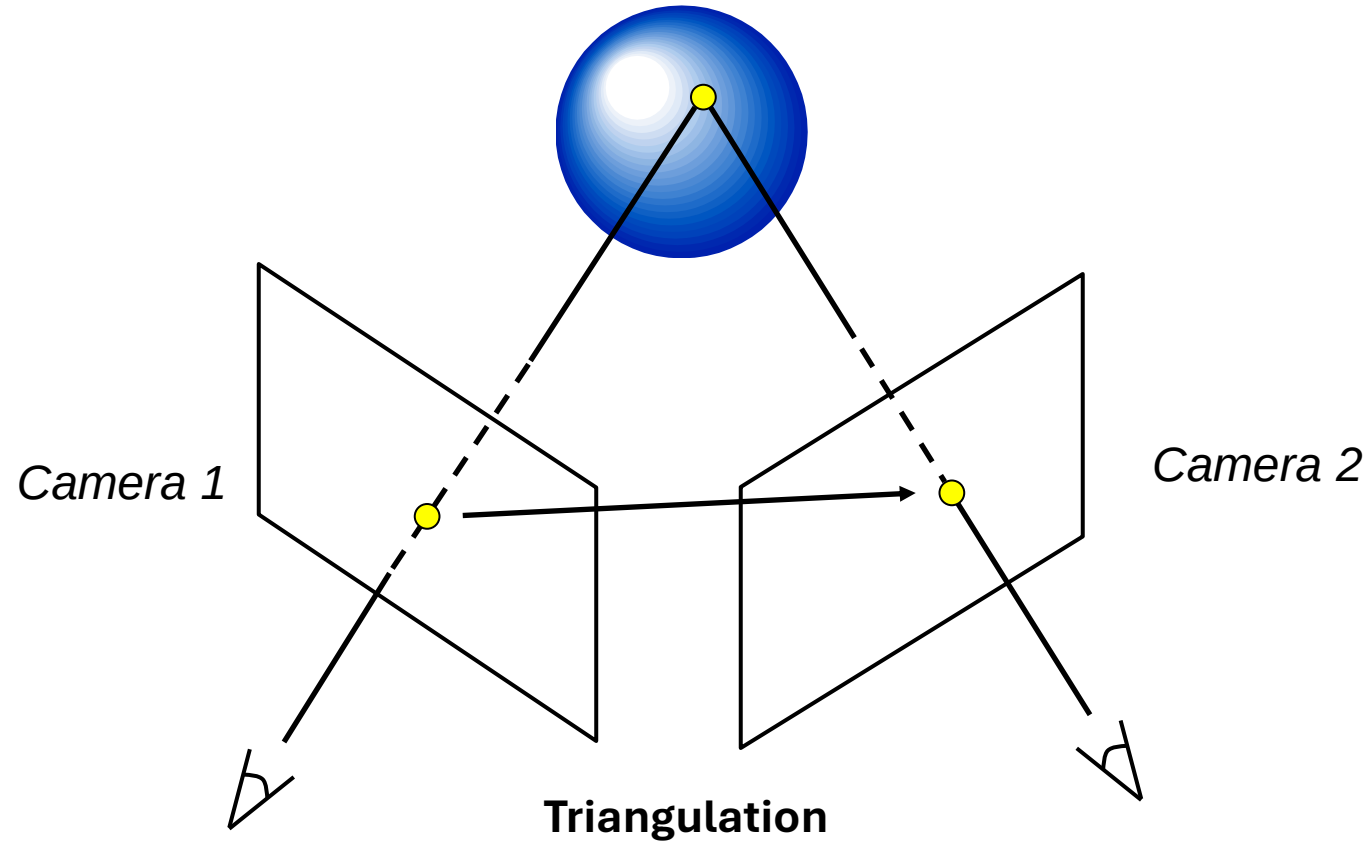
- Computation of corresponding points,
- Triangulation

# Stereopsi



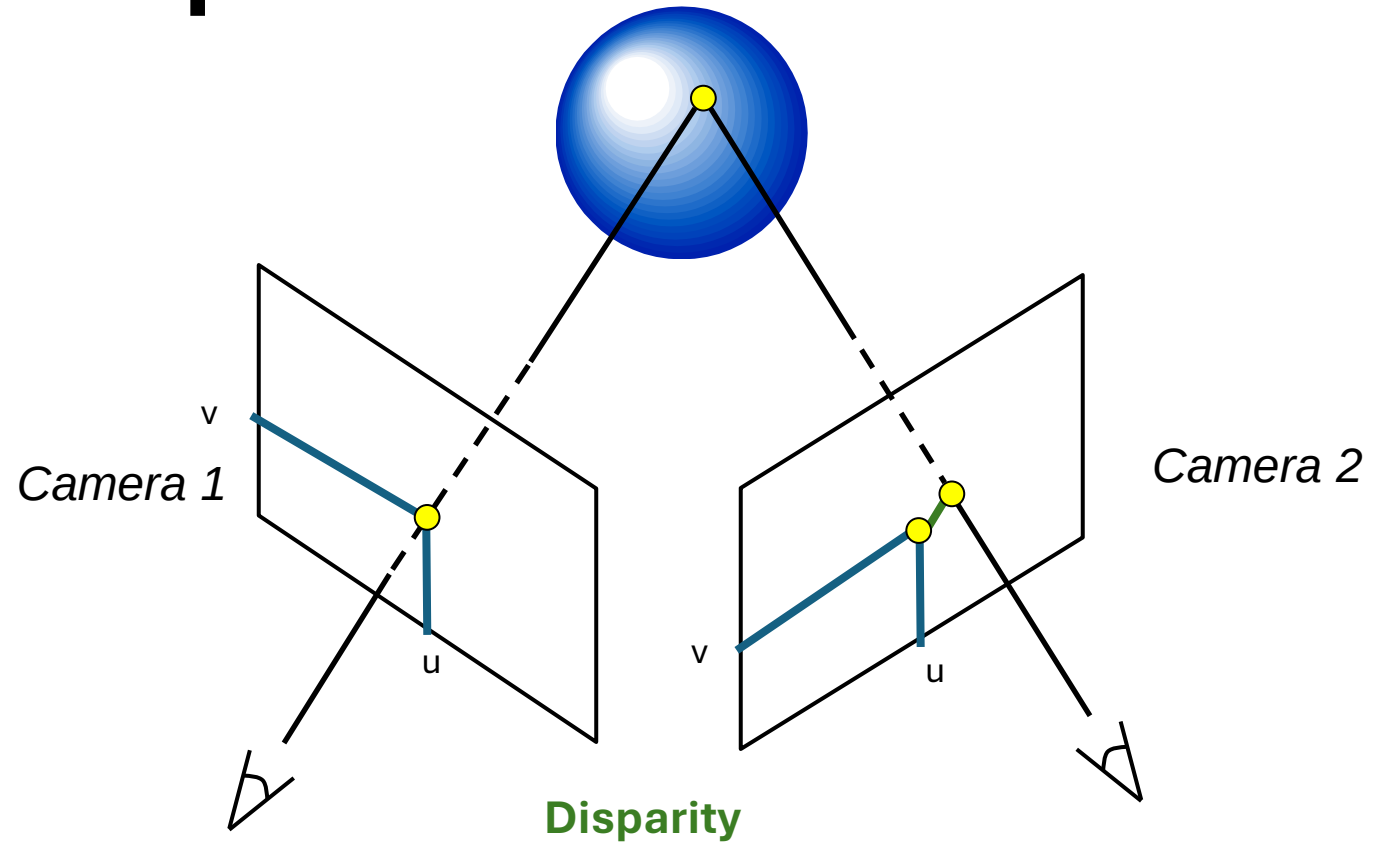
**Conjugate points or Corresponding points**

# Stereopsi



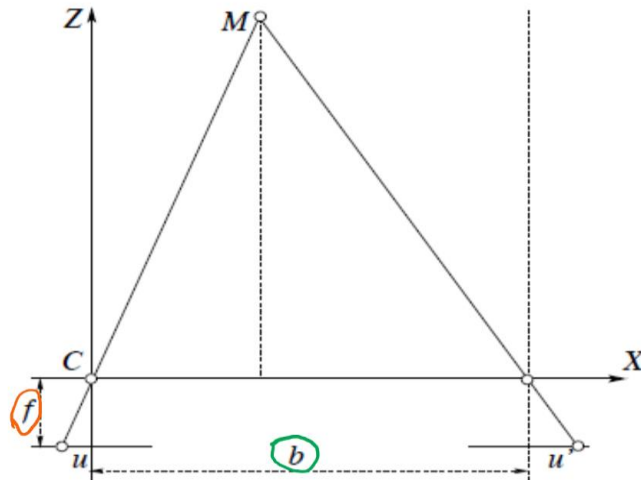


# Stereopsi



# TRIANGULATION

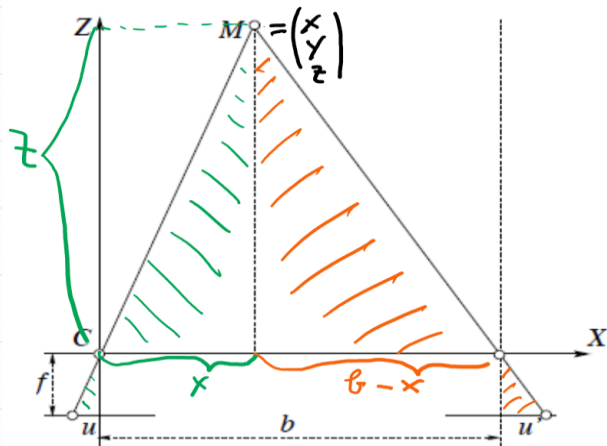
GIVEN TWO CAMERAS WITH THE SAME ORIENTATION (i.e., PARALLEL CAMERAS)  
AND OVERLAPPING FRAMES  $\Rightarrow$  THE DISPARITY IS WITH ONLY HORIZONTAL CONTRIBUTION



$f$  IS THE FOCAL LENGTH

$b$  IS THE DISTANCE BETWEEN THE  
VIEWPOINTS OR **BASELINE**

# TRIANGULATION



THE WORLD FRAME IS FIXED ON THE LEFT CAMERA

WE CAN DEFINE THE FOLLOWING EQUATIONS

$$\begin{cases} \frac{f}{z} = -\frac{u}{x} \\ \frac{f}{z} = \frac{u'}{b-x} \end{cases}$$



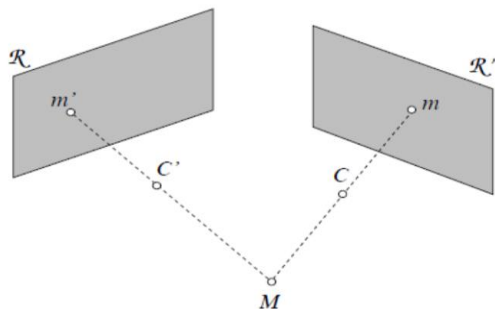
$$z = \frac{bf}{u' - u}$$

TO ESTIMATE THE DEPTH  $z$  WE NEED TO KNOW (i) THE GEOMETRY OF STEREO SETUP ( $b$  AND  $f$ ), AND (ii) THE DISPARITY ( $u' - u$ )

**NOTE:** THE DISPARITY ( $u' - u$ ) IS INVERSELY PROPORTIONAL TO THE DEPTH  $z$

# TRIANGULATION

## METODO LINEAR-EIGEN

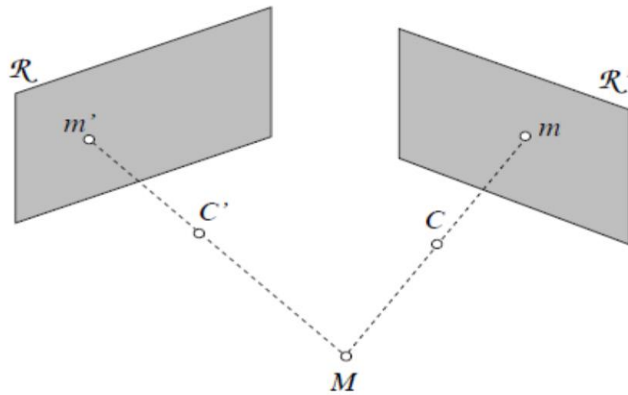


FOR THE MORE GENERIC CASE WE HAVE TWO ARBITRARY CAMERAS AND THE RESPECTIVE PERSPECTIVE MATRICES  $P$  AND  $P'$

GIVEN AN IMAGE POINT  $\underline{m} = \begin{pmatrix} u \\ v \\ 1 \end{pmatrix}$  THAT IS THE PROJECTION OF THE 3D POINT  $\underline{M}$  (UNKNOWN), WE HAVE

$$\begin{cases} p_1^T M - u p_3^T M = 0 \\ p_2^T M - v p_3^T M = 0 \end{cases} \Rightarrow \begin{cases} (p_1 - u p_3)^T M = 0 \\ (p_2 - v p_3)^T M = 0 \end{cases}$$

# TRIANGULATION



IN MATRIX FORM :

$$\begin{bmatrix} (p_1 - u p_3)^T \\ (p_2 - v p_3)^T \end{bmatrix}^{2 \times 4} M^{4 \times 1} = O_{2 \times 1}$$

ONE POINT GENERATES TWO EQUATIONS

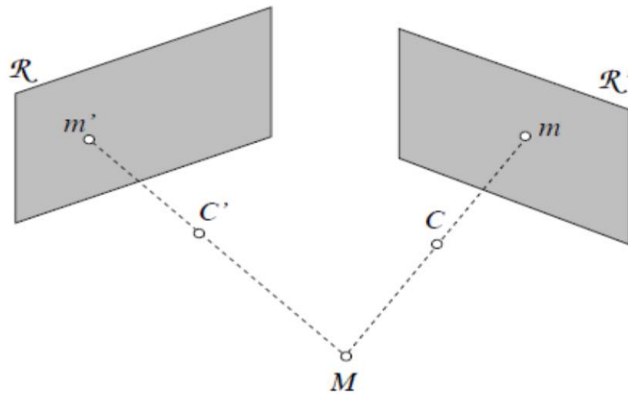
WE CAN CONSIDER  $m' = \begin{pmatrix} u' \\ v' \\ 1 \end{pmatrix}$  AND  $p'$ :

$$\begin{bmatrix} (p_1 - u p_3)^T \\ (p_2 - v p_3)^T \\ (p_1' - u' p_3')^T \\ (p_2' - v' p_3')^T \end{bmatrix}^{4 \times 4} M^{4 \times 1} = O_{4 \times 1}$$

THE SOLUTION IS OBTAINED FROM THIS HOMOGENEOUS LINEAR SYSTEM



# TRIANGULATION



IN MATRIX FORM :

$$\begin{bmatrix} (p_1 - u p_3)^T \\ (p_2 - v p_3)^T \end{bmatrix}^{\frac{2 \times 4}{1}} M^{\frac{4 \times 1}{1}} = O_{2 \times 1}$$

ONE POINT GENERATES TWO EQUATIONS

WE CAN CONSIDER  $m' = \begin{pmatrix} u' \\ v' \\ 1 \end{pmatrix}$  AND  $P'$ :

NOTE: TWO VIEWS ARE SUFFICIENT  
TO COMPLETE OUR LINEAR SYSTEM  
(i.e. 4 EQUATIONS FOR 4 UNKNOWN)  
DUE TO THE NOISE IT IS BETTER  
TO SOLVE THE LINEAR SYSTEM  
WITH LEAST SQUARED STRATEGY  
MORE VIEWS SHOULD BE CONSIDERED  
ADDING NEW LINES

$$\begin{bmatrix} (p_1 - u p_3)^T \\ (p_2 - v p_3)^T \\ (p_1' - u' p_3')^T \\ (p_2' - v' p_3')^T \end{bmatrix}^{\frac{4 \times 4}{1}} M^{\frac{4 \times 1}{1}} = O_{4 \times 1}$$

THE SOLUTION IS OBTAINED FROM THIS HOMOGENEOUS LINEAR SYSTEM